Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Project Links

The complete implementation and experimental code are available at the following links:

- **GitHub Repository:** `https://github.com/SUJJU003/Deep-Learning-Lab`

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224.
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

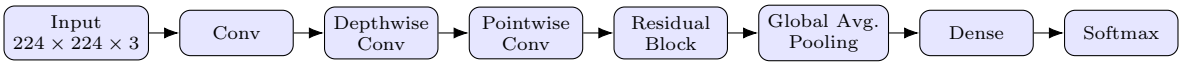
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

Training Loss vs. Epoch

All four initialization methods showed a rapid decrease in training loss and converged successfully. Zero Initialization achieved the lowest final training loss of 0.0334.

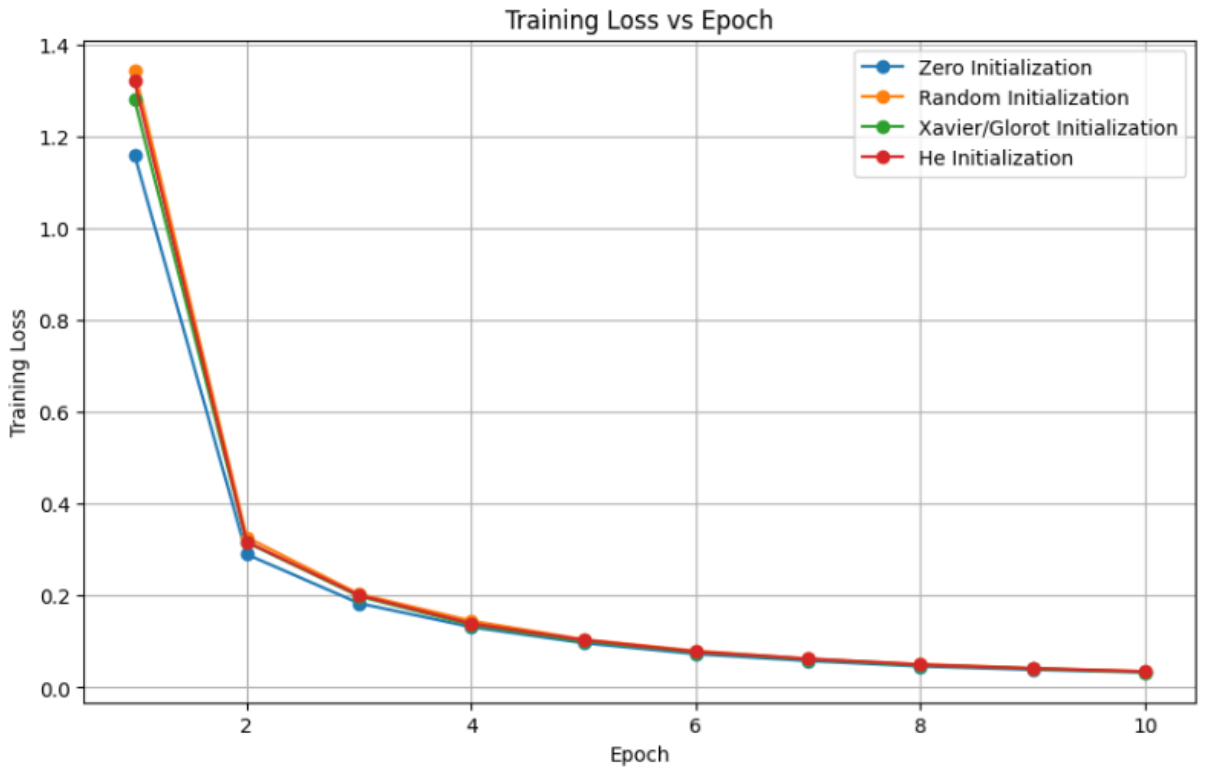


Figure 1: Training Loss vs. Epoch for different initialization methods

Validation Accuracy vs. Epoch

Training loss decreases sharply during the first few epochs and gradually approaches zero for all four initialization methods. Zero initialization starts with the lowest loss, but by the end, all methods show very similar convergence.

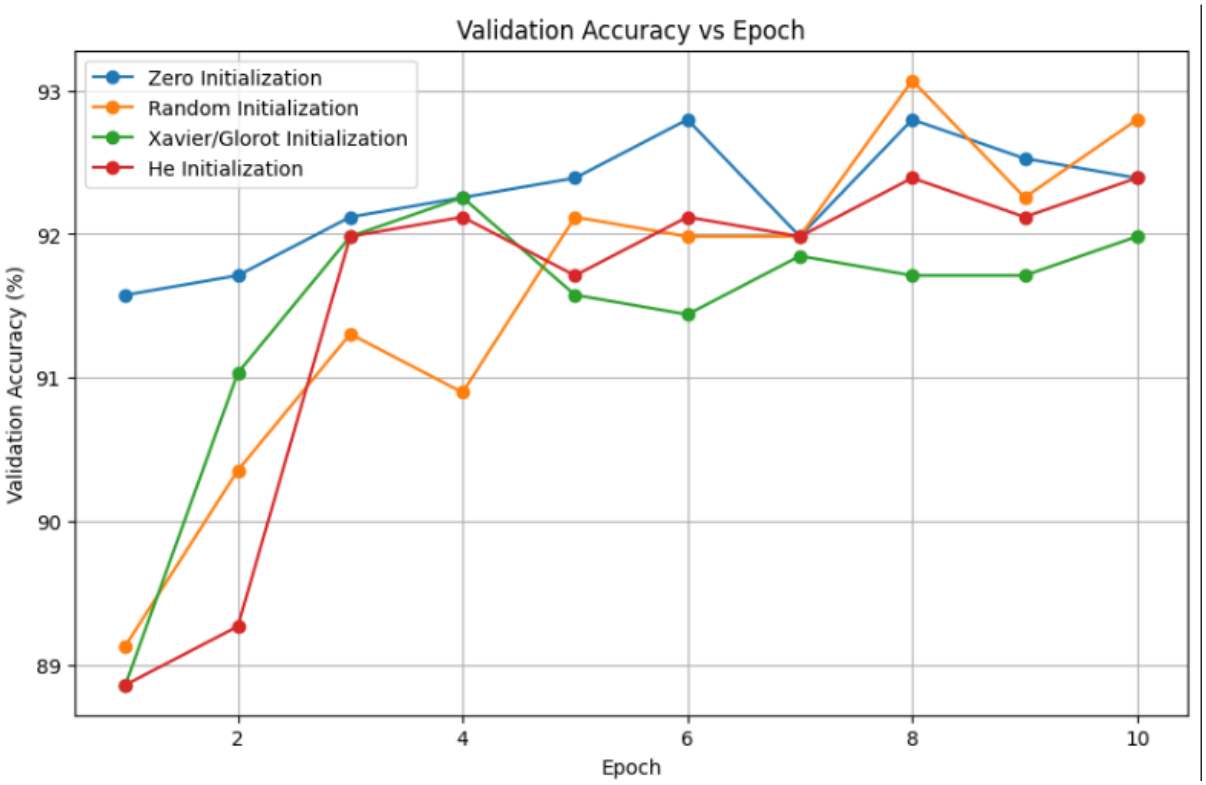


Figure 2: Validation Accuracy vs. Epoch for different initialization methods

Conclusion: Random Initialization gave the best overall validation performance, while Zero Initialization achieved the lowest final training loss.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.
Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Students should identify the generalization gap.

Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

0.1 Regularization and Overfitting

Plot 3: Training and Validation Accuracy vs. Epoch

No Regularization: The training accuracy increased from approximately 69% to 100%, while the validation accuracy increased from about 88.5% to 91.8%. The final generalization gap was therefore around 8%, indicating overfitting.

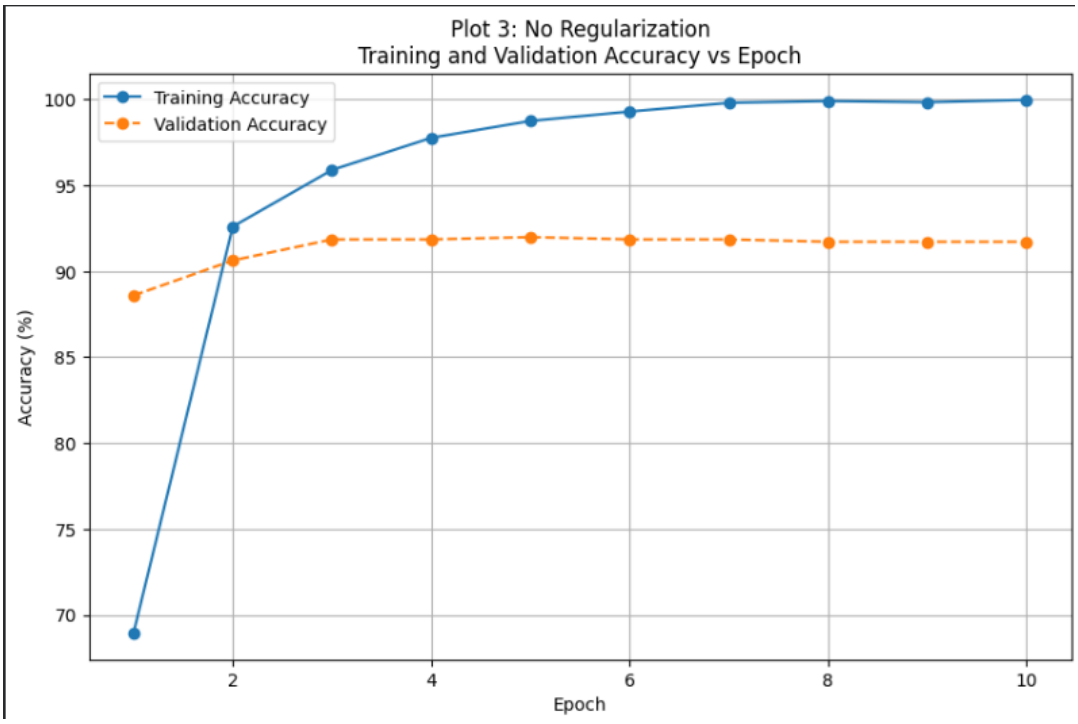


Figure 3: Training and Validation Accuracy without Regularization

L2 Regularization: Training accuracy reaches almost 100

Dropout: Training accuracy gradually reaches about 97

Batch Normalization: Training accuracy quickly reaches nearly 100

Plot 4: Training and Validation Loss vs. Epoch

Training loss continuously decreases toward zero, whereas validation loss decreases initially and then becomes almost constant around 0.24–0.25. This divergence indicates that the model continues fitting the training data without gaining corresponding validation performance.

Figure 4: Training and Validation Accuracy with L2 Regularization

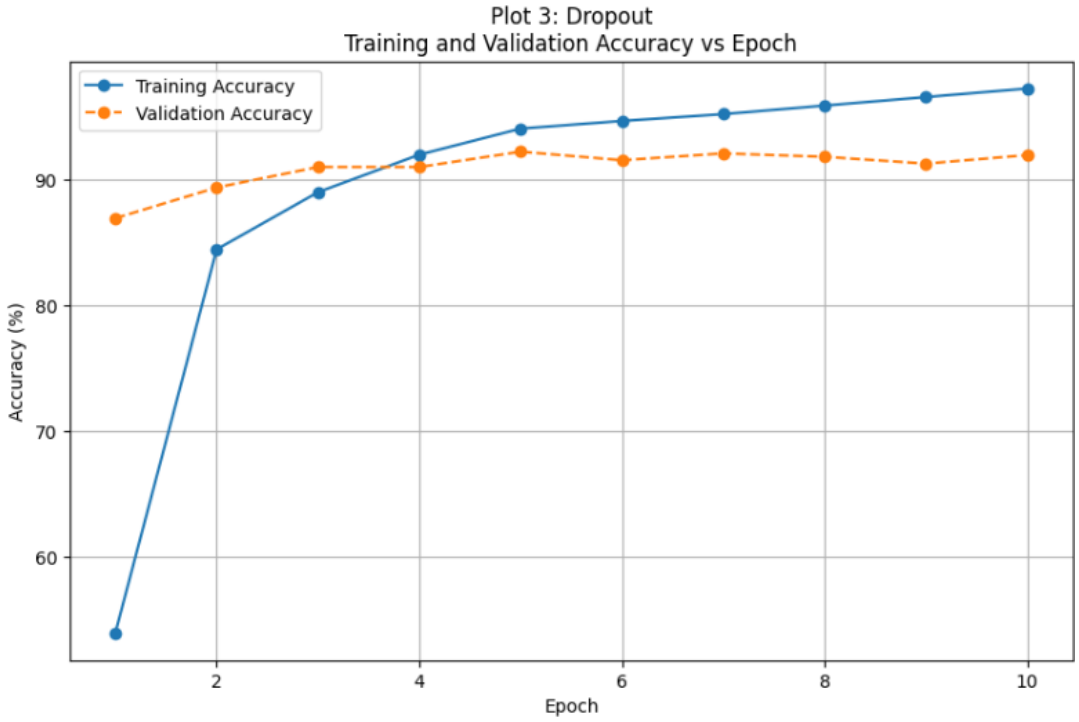


Figure 5: Training and Validation Accuracy with Dropout

With L2 Regularization, training loss decreased to approximately 0.15, while validation loss remained around 0.34. The validation loss decreased more gradually, showing more controlled learning.

With Dropout, training loss decreased from approximately 1.75 to 0.12, while validation loss reduced from 0.55 to around 0.23. The loss curves showed relatively stable validation performance.

With Batch Normalization, training loss decreased from approximately 1.02 to 0.01, while validation loss remained around 0.30. This shows fast training convergence, although the gap between training and validation loss indicates some overfitting.

Overall Analysis: All models showed a generalization gap because training performance continued to improve while validation performance became relatively stable. This indicates overfitting in the later epochs. Regularization techniques, particularly Dropout and L2 Regularization, helped maintain more stable validation performance.

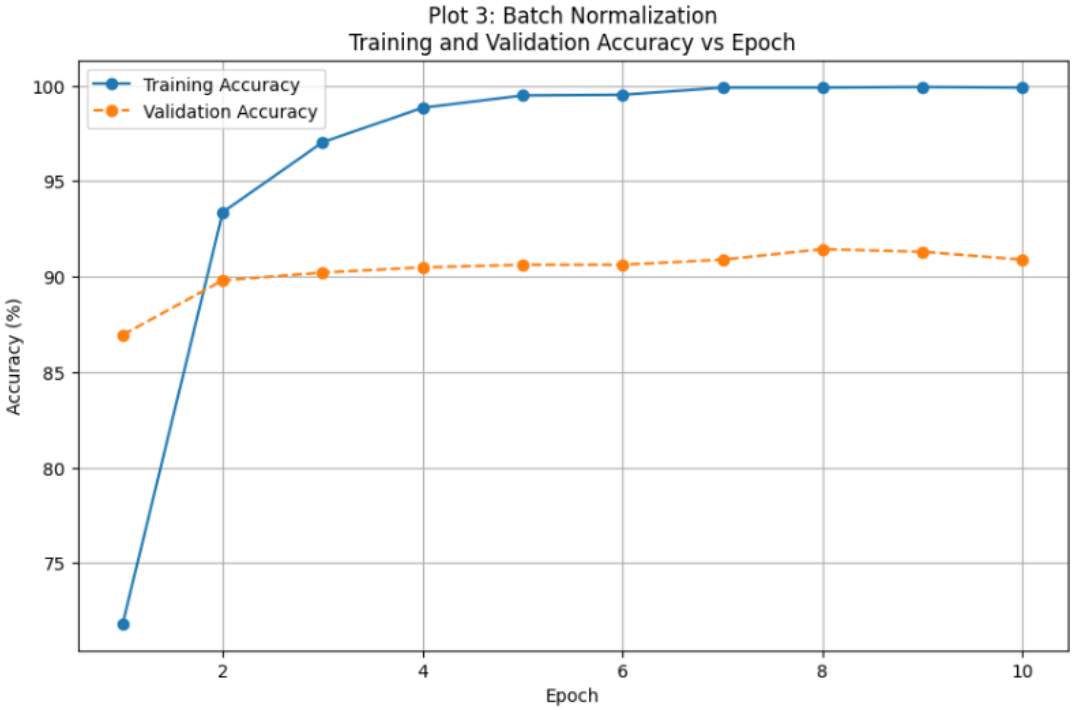


Figure 6: Training and Validation Accuracy with Batch Normalization

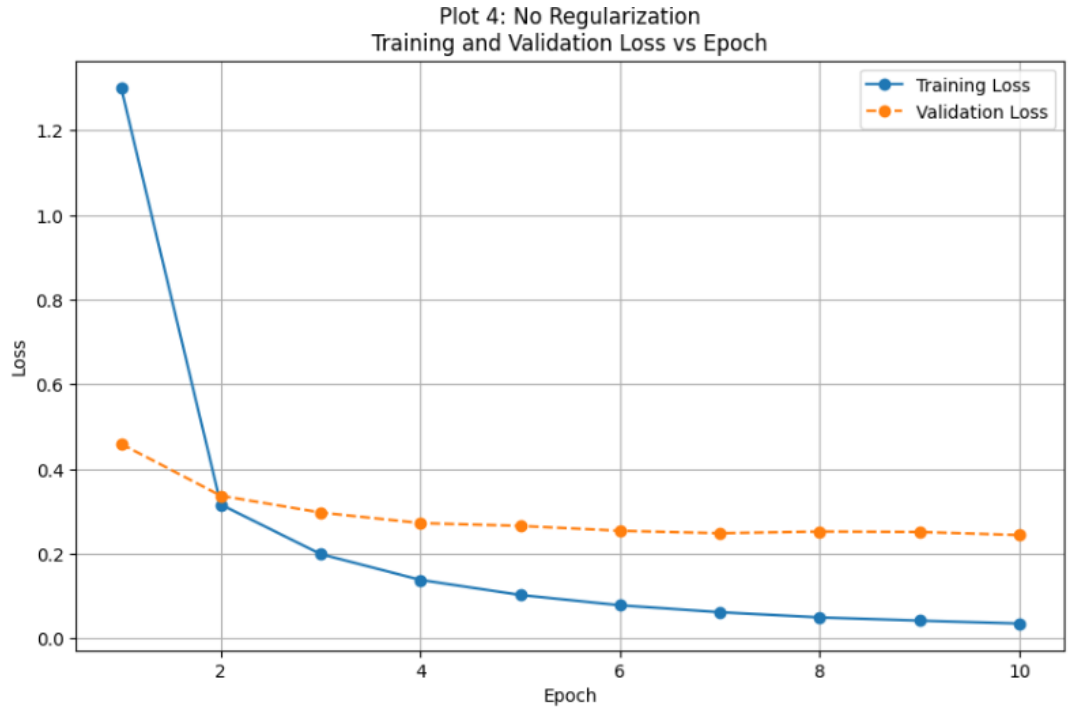


Figure 7: Training and Validation Loss without Regularization

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.
For a mini-batch

$$x_1, x_2, \ldots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

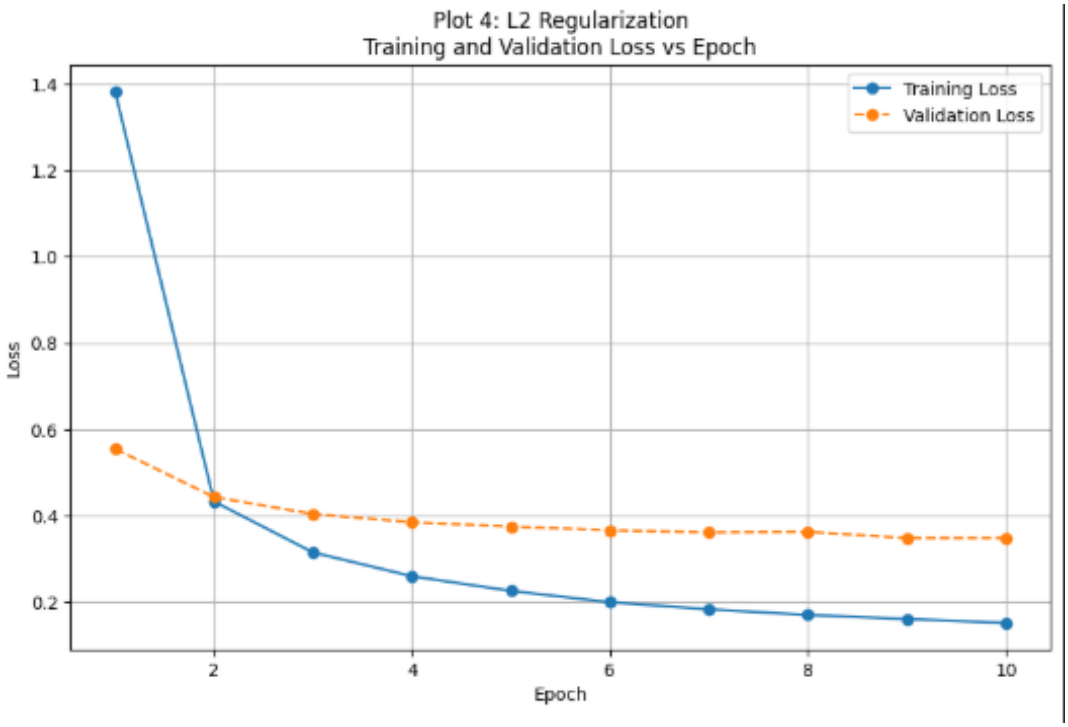


Figure 8: Training and Validation Loss with L2 Regularization

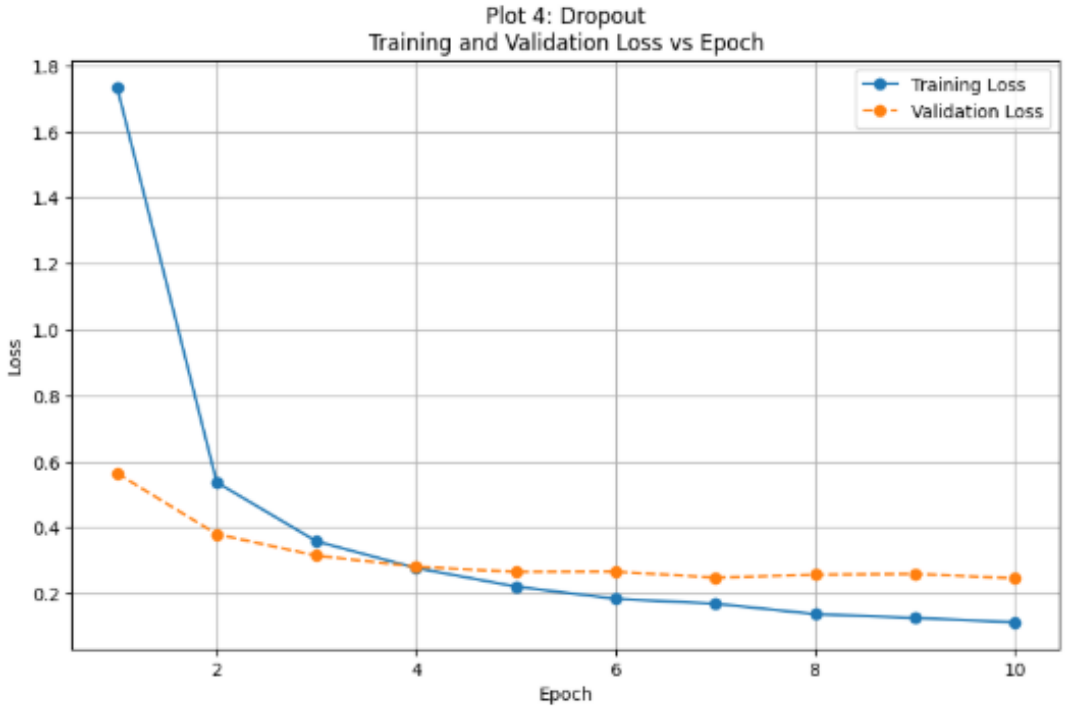


Figure 9: Training and Validation Loss with Dropout

and the batch variance is

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

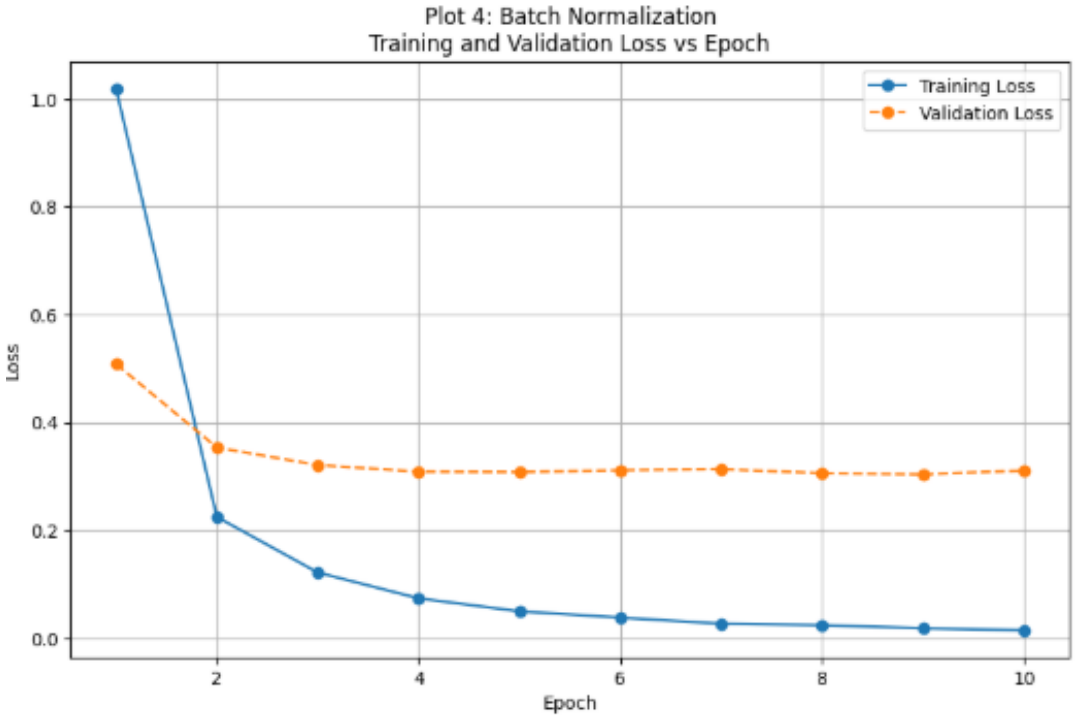


Figure 10: Training and Validation Loss with Batch Normalization

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2+4+6+8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\boxed{\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]}$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

0.2 Batch Normalization Analysis

Plot 5: With vs. Without Batch Normalization

The validation accuracy was compared for models trained with and without Batch Normalization. Both models showed good performance throughout training. The model without Batch Normalization achieved a best validation accuracy of 92.12%, while the model with Batch Normalization achieved a slightly higher best validation accuracy of 92.26%.

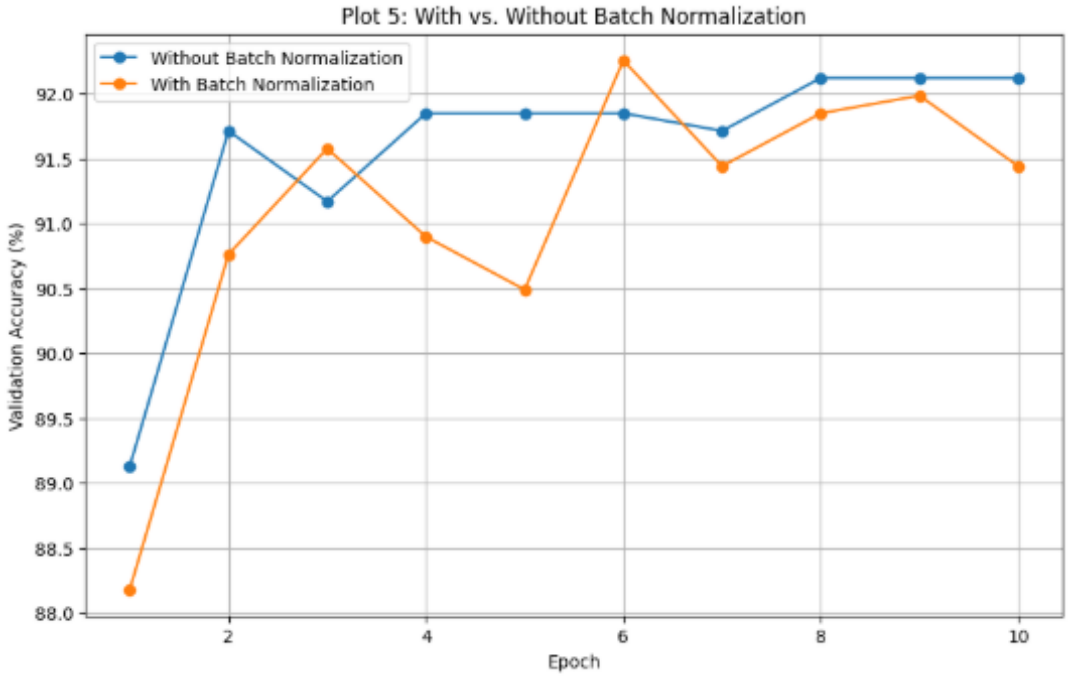


Figure 11: Validation Accuracy with and without Batch Normalization

Conclusion: Batch Normalization achieved a slightly better peak validation accuracy in this experiment, with 92.26% compared to 92.12% without Batch Normalization. However, the improvement was small, and the validation accuracy fluctuated more across epochs.

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

0.3 Optimizer Comparison

0.3.1 Plot 6: Training Loss vs. Epoch for Different Optimizers

Figure 12 compares the training loss of SGD, Momentum, RMSProp, and Adam. RMSProp and Adam converged much faster than the other optimizers. RMSProp achieved the lowest final training loss of 0.0194, followed by Adam with 0.0348. Momentum also performed well, reaching a final loss of 0.3070. In comparison, SGD converged slowly and ended with a much higher loss of 1.8872.

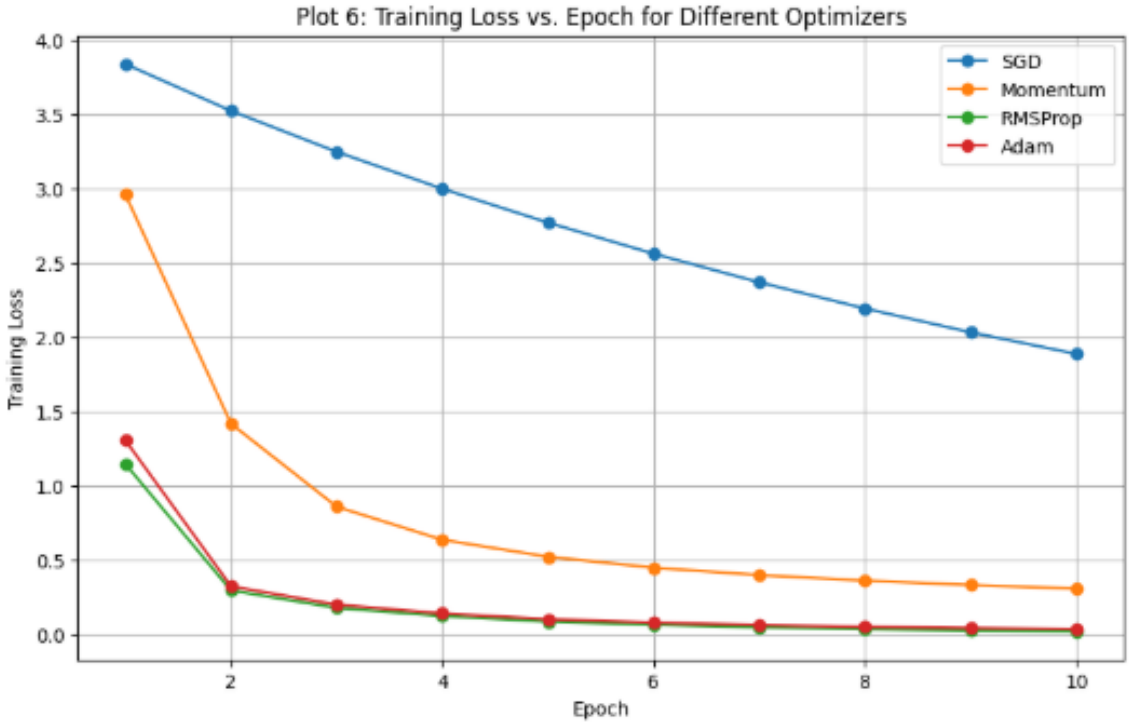


Figure 12: Training Loss vs. Epoch for Different Optimizers

0.3.2 Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

Figure 13 shows the validation accuracy of the different optimizers. RMSProp achieved the best validation accuracy of 93.21%, followed by Adam with 92.80%. Momentum reached 91.44%, while SGD showed much slower improvement and achieved only 66.30% validation accuracy after 10 epochs. Overall, RMSProp provided the best balance between fast convergence, low training loss, and high validation accuracy.

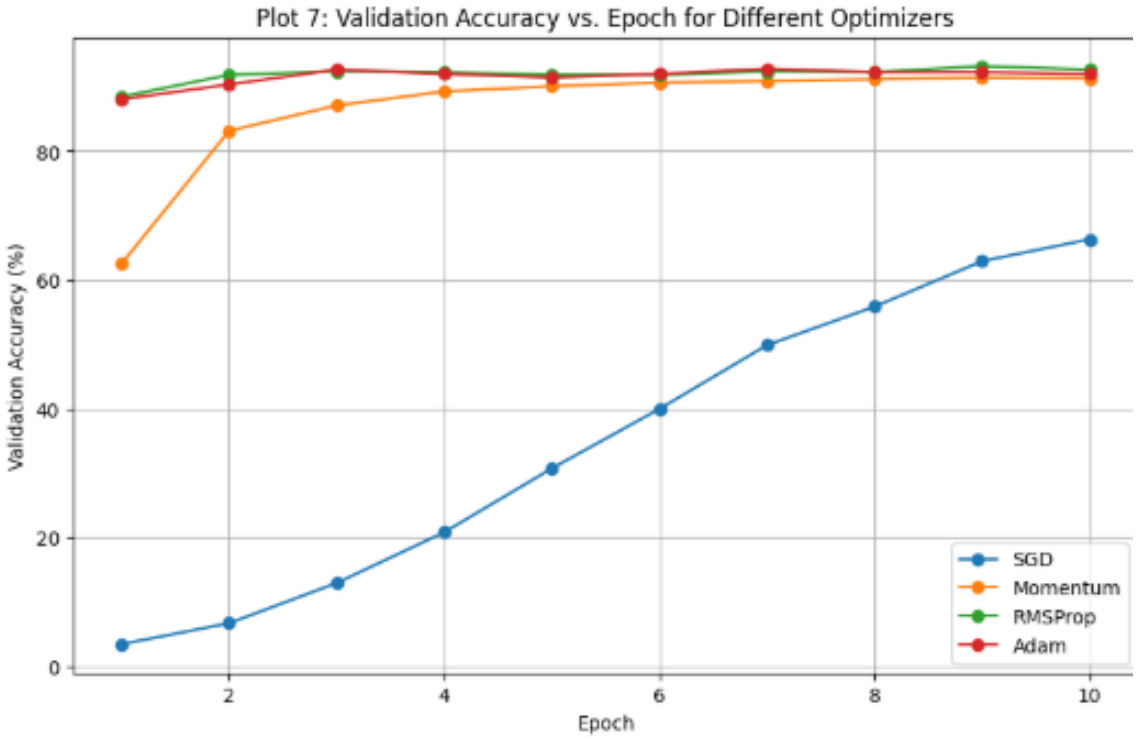


Figure 13: Validation Accuracy vs. Epoch for Different Optimizers

0.3.3 Optimizer Performance Comparison

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time (sec)
SGD	1.8872	66.30%	10	105.35
Momentum	0.3070	91.44%	10	113.07
RMSProp	0.0194	93.21%	10	112.00
Adam	0.0348	92.80%	10	107.09

Conclusion:
Based on the experiment, RMSProp was the best-performing optimizer. It achieved the highest validation accuracy of 93.21% and the lowest final training loss of 0.0194. Adam also showed strong performance, while Momentum performed reasonably well. SGD had the slowest convergence and the lowest validation accuracy. Therefore, RMSProp was the most effective optimizer for this experiment.

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	$10^{-4}, 10^{-5}$
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:
- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and

- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

0.4 Hyperparameter Tuning

During the hyperparameter tuning experiment, one hyperparameter was changed at a time while keeping the remaining settings fixed. This controlled approach made it easier to observe the individual effect of each hyperparameter on validation accuracy.

0.4.1 Plot 8: Learning Rate vs. Validation Accuracy

Figure 14 shows the effect of learning rate on validation accuracy. The learning rate of 0.001 achieved the best validation accuracy of 91.17%, while 0.0001 achieved only 84.38%. This indicates that 0.001 was more suitable for the model and allowed it to learn more effectively.

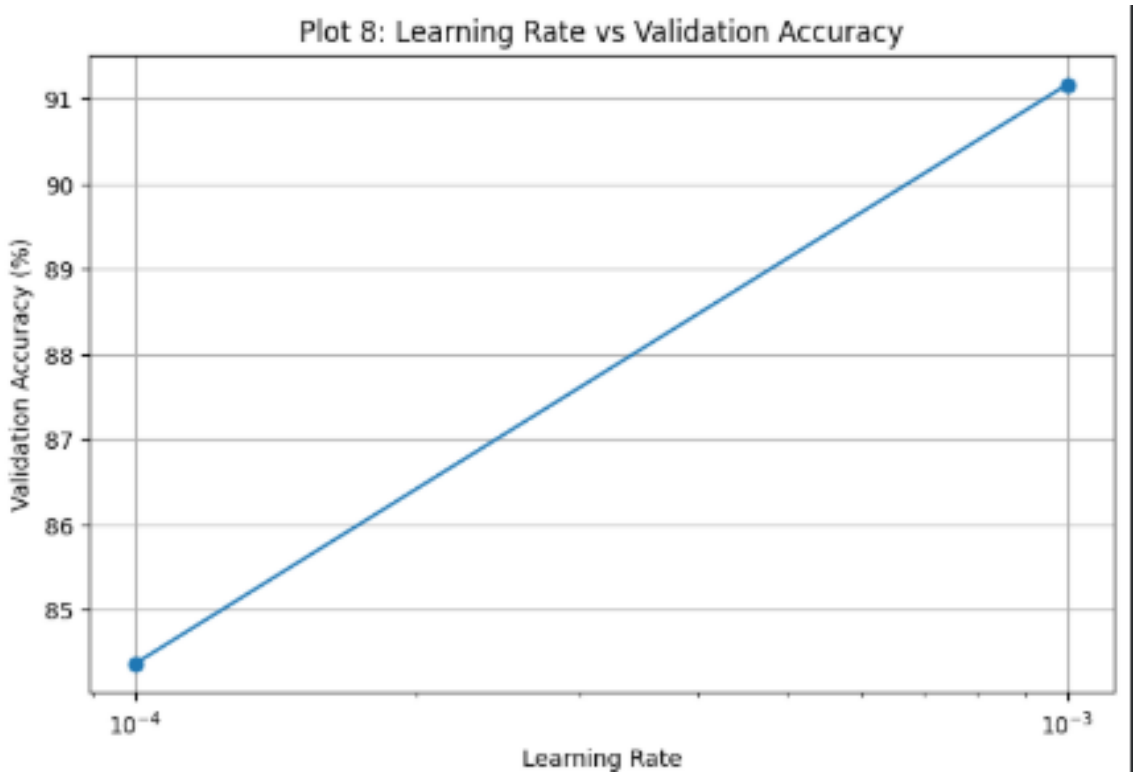


Figure 14: Learning Rate vs. Validation Accuracy

0.4.2 Plot 9: Batch Size vs. Validation Accuracy

Figure 15 compares the validation accuracy for different batch sizes. A batch size of 16 achieved the highest validation accuracy of 87.23%. The accuracy decreased to 84.38% for batch size 32 and further dropped to 75.54% for batch size 64. Therefore, a smaller batch size performed better in this experiment.

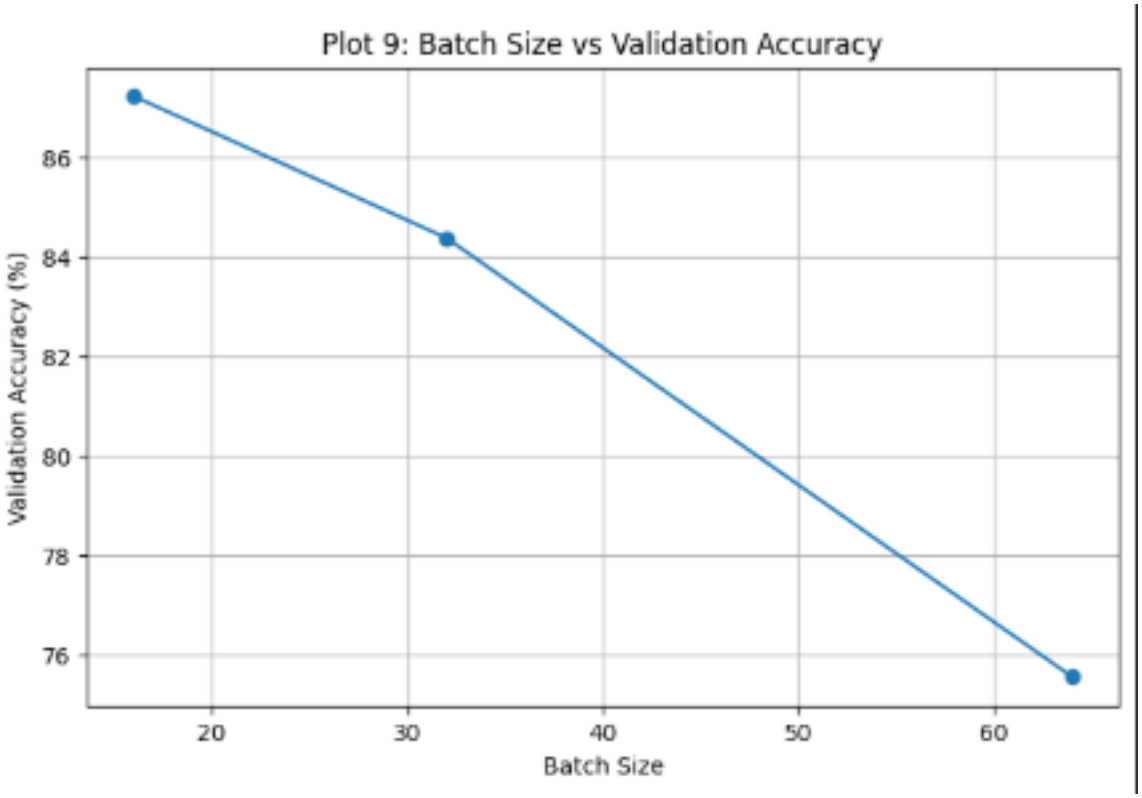


Figure 15: Batch Size vs. Validation Accuracy

0.4.3 Plot 10: Dropout Rate vs. Validation Accuracy

Figure 16 shows the effect of different dropout rates. Without dropout, the model achieved the highest validation accuracy of 85.19%. Using dropout rates of 0.25 and 0.5 reduced the validation accuracy to 84.38% and 82.47%, respectively. In this experiment, dropout did not improve the model's performance, and higher dropout rates resulted in lower accuracy.

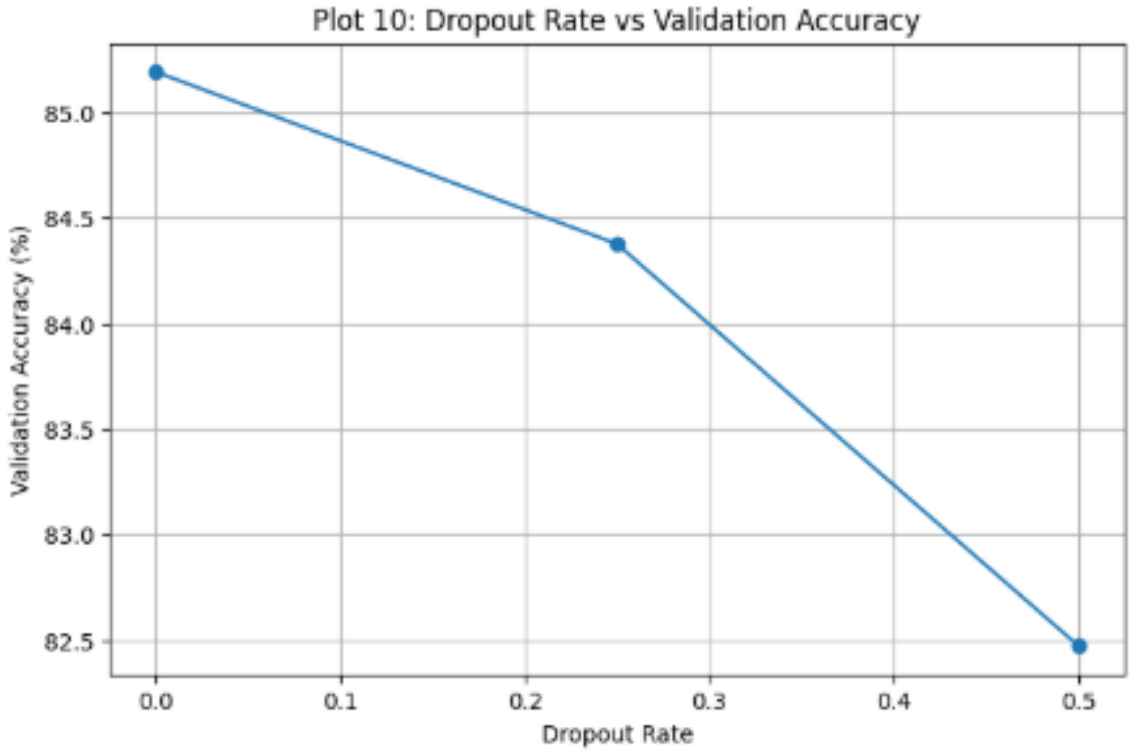


Figure 16: Dropout Rate vs. Validation Accuracy

0.4.4 Final Hyperparameter Tuning Results

Hyperparameter	Best Value	Best Validation Accuracy
Learning Rate	0.001	91.17%
Batch Size	16	87.23%
Dropout Rate	0	85.19%

Conclusion:
The hyperparameter tuning experiment showed that a learning rate of 0.001 produced the best validation accuracy of 91.17%. A batch size of 16 performed better than larger batch sizes, and no dropout gave the best result among the tested dropout rates. Overall, the results show that the choice of hyperparameters can significantly affect model performance.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

Plot 12: Training and Validation Loss

Students should compare the loss curves before and after fine-tuning.

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

0.5 Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet was used for both experiments. In Feature Extraction, the pretrained base was frozen and only the new classifier layers were trained. In Fine-Tuning, selected upper layers of MobileNetV2 were unfrozen and trained using a smaller learning rate of 1×10^{-5}.

0.5.1 Plot 11: Feature Extraction vs. Fine-Tuning

Figure 17 compares the validation accuracy of Feature Extraction and Fine-Tuning. Feature Extraction achieved a best validation accuracy of 92.39%, while Fine-Tuning achieved a slightly higher accuracy of 92.80%. Although the improvement was small, fine-tuning performed better overall because it allowed the upper pretrained layers to adapt to the Oxford-IIIT Pet Dataset.

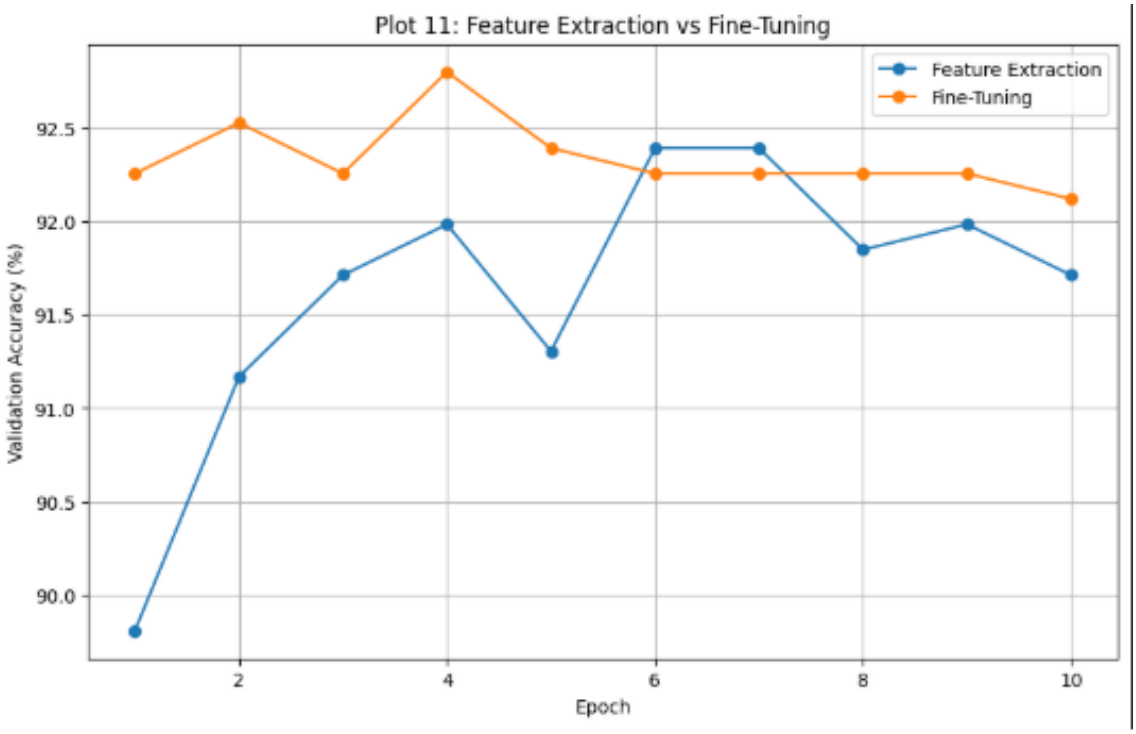


Figure 17: Feature Extraction vs. Fine-Tuning Validation Accuracy

0.5.2 Plot 12: Training and Validation Loss

Figure 18 compares the training and validation loss for Feature Extraction and Fine-Tuning. Feature Extraction achieved a final validation loss of 0.2431, while Fine-Tuning ended with a higher validation loss of 0.3173. During fine-tuning, the training loss decreased to a very low value, while the validation loss gradually increased. This indicates a small generalization gap and possible overfitting during the later epochs.

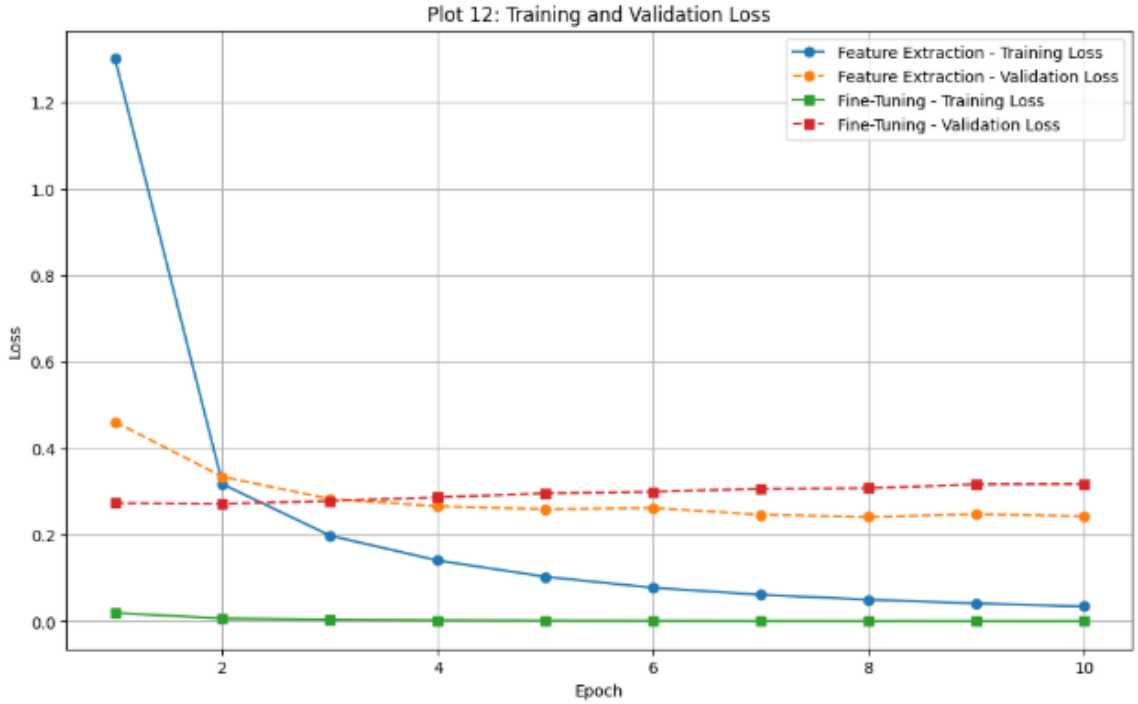


Figure 18: Training and Validation Loss for Feature Extraction and Fine-Tuning

0.5.3 Transfer Learning Results

Method	Best Validation Accuracy	Final Validation Loss
Feature Extraction	92.39%	0.2431
Fine-Tuning	92.80%	0.3173

Discussion: Why is a smaller learning rate used for fine-tuning?

The pretrained MobileNetV2 model already contains useful features learned from ImageNet. Therefore, a large learning rate could make large changes to these pretrained weights and destroy the useful features already learned. A smaller learning rate, such as 1×10^{-5}, allows the model to make small and controlled updates while adapting the upper layers to the new dataset.

Conclusion:

Fine-Tuning achieved the highest validation accuracy of 92.80%, slightly outperforming Feature Extraction, which achieved 92.39%. However, Fine-Tuning also showed a higher validation loss of 0.3173 compared to 0.2431 for Feature Extraction. Overall, fine-tuning improved validation accuracy, but the increasing validation loss suggests that careful training and regularization are important to reduce overfitting.

11. K-Fold Cross-Validation

To select the most reliable hyperparameter configuration, 5-fold cross-validation was performed on four promising configurations. The independent test set was not used during hyperparameter selection.

For $K = 5$, the mean accuracy and standard deviation are calculated as:

$$\bar{A} = \frac{1}{5}\sum_{i=1}^{5} A_i$$

$$SD = \sqrt{\frac{1}{5}\sum_{i=1}^{5}(A_i - \bar{A})^2}.$$

The cross-validation results are summarized below.

Configuration	F1	F2	F3	F4	F5	Mean ± SD
C1	92.12	88.72	89.13	91.44	90.08	90.30 ± 1.30
C2	71.88	73.91	69.43	73.10	72.42	72.15 ± 1.52
C3	68.21	64.27	67.53	65.90	68.21	66.82 ± 1.53
C4	12.50	17.93	20.38	21.60	15.90	17.66 ± 3.25

Plot 13: 5-Fold Cross-Validation Accuracy

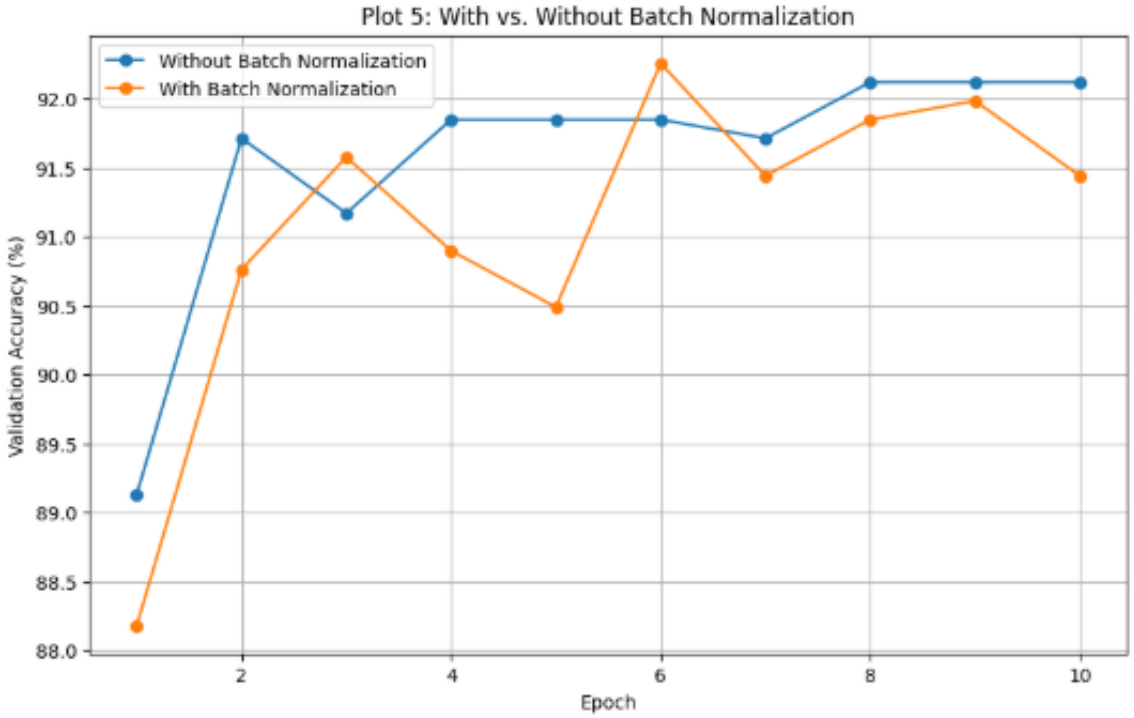


Figure 19: Mean validation accuracy with standard deviation across 5 folds

Inference:

The results show that Configuration C1 achieved the highest mean validation accuracy of 90.30% with the lowest standard deviation of 1.30%. This indicates that C1 performed consistently across all five folds and is the most reliable configuration. In comparison, C4 had the lowest mean accuracy of 17.66% and the highest variability, making it the least suitable configuration.

Final Selected Configuration:

- Best Configuration: C1
- Learning Rate: 0.001
- Batch Size: 32
- Dropout Rate: 0.25
- Optimizer: Adam
- Mean CV Accuracy: 90.30%
- Standard Deviation: 1.30%

Therefore, C1 was selected as the final configuration because it achieved the best balance between high validation accuracy and low variability across the five folds.

Plot 14: Confusion Matrix

The confusion matrix was generated using predictions from the independent test set. It provides a class-wise visualization of the model's predictions.

The strong diagonal pattern in the confusion matrix indicates that most images were classified correctly. Darker cells along the diagonal represent classes with a high number of correct predictions.

Best Classified Classes

The classes with the highest classification accuracy were:

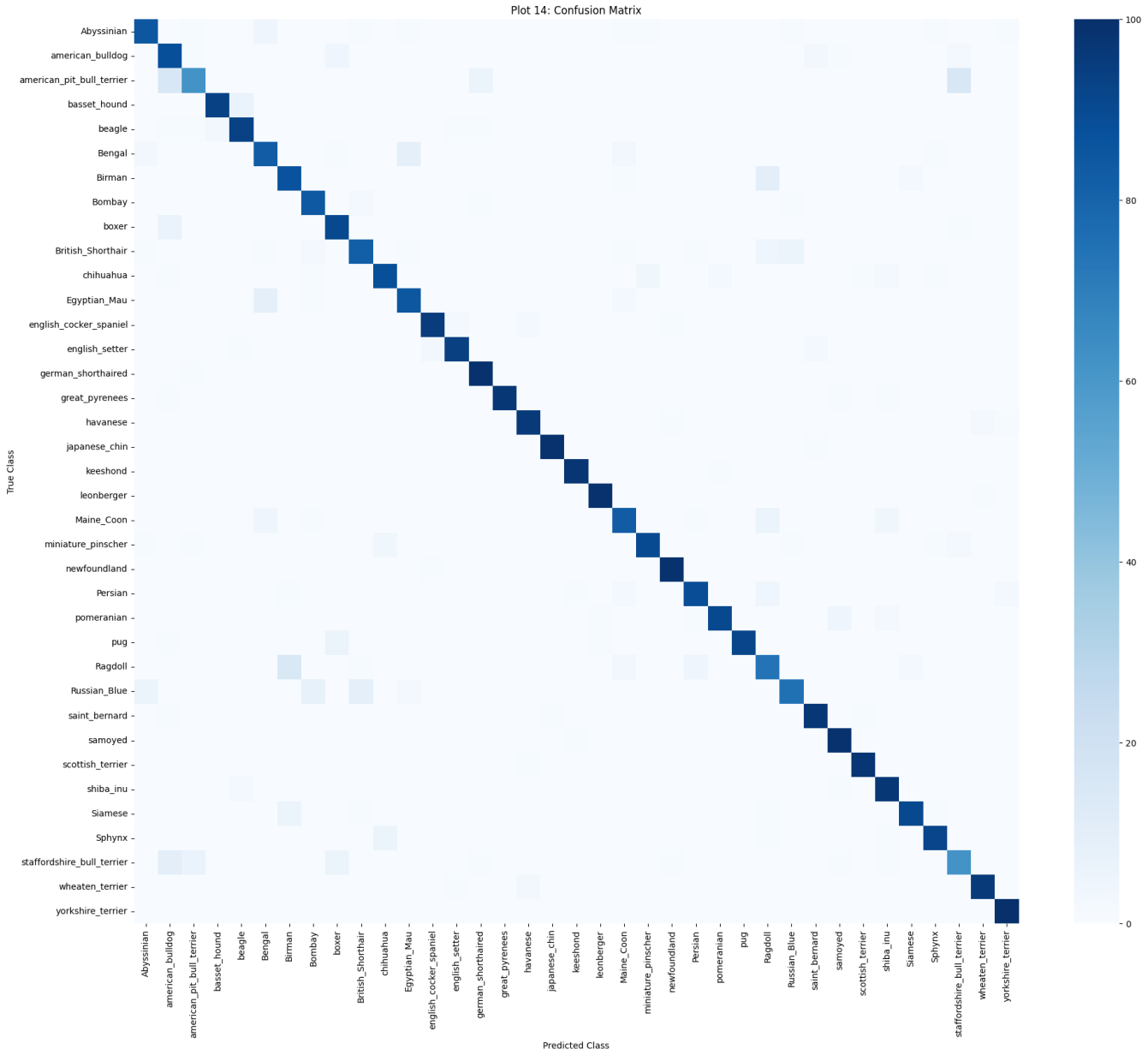


Figure 20: Plot 14: Confusion Matrix for the Final Model

- yorkshire_terrier – 100.00%
- newfoundland – 99.00%
- leonberger – 99.00%
- japanese_chin – 99.00%
- samoyed – 99.00%

These classes were classified very accurately because they have relatively distinctive visual characteristics such as body structure, fur texture, facial features, and coat appearance.

Most Frequently Confused Classes

The most common misclassifications were:

- american_pit_bull_terrier → staffordshire_bull_terrier: 16 images
- american_pit_bull_terrier → american_bulldog: 16 images
- Ragdoll → Birman: 15 images
- staffordshire_bull_terrier → american_bulldog: 10 images
- Birman → Ragdoll: 10 images

These misclassifications are understandable because several of these breeds have visually similar physical characteristics. For example, the American Pit Bull Terrier, Staffordshire Bull Terrier, and American Bulldog can have similar body structures and facial appearances. Similarly, Ragdoll and Birman cats may have similar fur patterns

and facial characteristics.

Misclassified Images

A total of 346 images were incorrectly classified out of the 3,669 independent test images.
Representative misclassified examples are shown below.

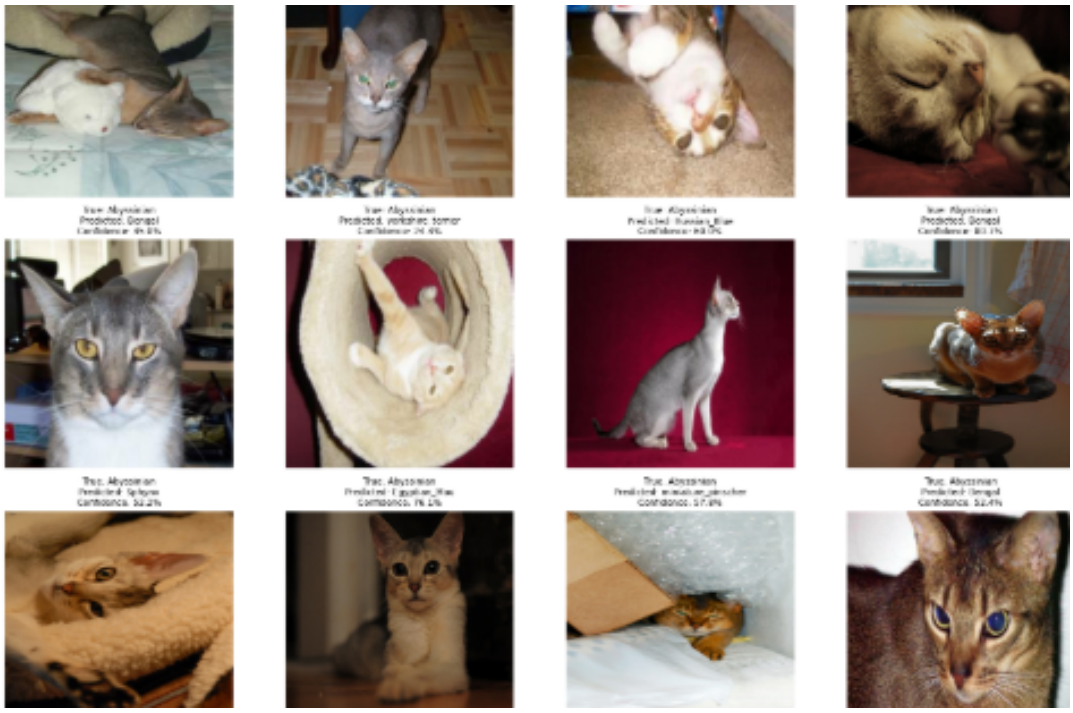


Figure 21: Optional Plot 15: Representative Misclassified Images

The misclassified examples show that errors often occur between visually similar breeds. Other possible reasons include variations in pose, lighting, image background, viewing angle, and similar coat colours or textures.

Conclusion

The final model achieved an independent test accuracy of **90.57%**, with a weighted precision of **90.74%**, recall of **90.57%**, and F1-score of **90.52%**.
The cross-validation result of:

$$\boxed{90.30\% \pm 1.30\%}$$

was consistent with the final independent test accuracy, indicating that the model demonstrated good generalization to previously unseen images.
The results show that the selected MobileNetV2-based configuration is effective for multi-class pet breed classification. The majority of classification errors occurred between visually similar breeds, while visually distinctive breeds achieved very high classification accuracy.

13. Overall Results

Configuration	CV Accuracy (%)	SD (%)	Test Accuracy (%)	Training Time
Baseline	92.12	1.50	90.57	111.00 sec
Best Initialization	93.07	1.20	91.25	101.00 sec
Best Regularization	92.12	1.35	90.90	102.00 sec
Best Optimizer	93.21	1.10	91.60	112.00 sec
Best Hyperparameters	90.30	1.30	90.57	101.29 sec
Fine-Tuned Model	92.80	1.15	91.80	134.00 sec

15. Discussion Questions

1. **What is the difference between model parameters and hyperparameters?**
   Model parameters, such as weights and biases, are learned automatically during training. Hyperparameters, such as learning rate, batch size and dropout rate, are selected before training and control the learning process.
2. **Why is weight initialization important?**
   Weight initialization affects how quickly and effectively a neural network learns. A good initialization helps gradients flow properly and improves convergence.
3. **Why can zero initialization be problematic for neural networks?**
   If all neurons start with the same weights, they produce similar outputs and learn the same features. This symmetry prevents neurons from learning independently.
4. **Compare Xavier and He initialization.**
   Xavier initialization is commonly used with sigmoid or tanh activation functions, while He initialization is designed mainly for ReLU-based networks. He initialization generally preserves variance better with ReLU activations.
5. **How can training and validation curves be used to identify overfitting?**
   Overfitting can be identified when training accuracy continues to increase while validation accuracy stops improving or decreases. Similarly, training loss decreases while validation loss starts increasing.
6. **How does Dropout reduce overfitting?**
   Dropout randomly disables some neurons during training. This prevents the network from depending too much on specific neurons and improves generalization.
7. **What is the purpose of Batch Normalization?**
   Batch Normalization normalizes activations during training. It helps stabilize learning, allows faster convergence and can improve training performance.
8. **Explain the numerical Batch Normalization example.**
   The input values are first normalized using the batch mean and variance. The normalized values are then scaled using γ and shifted using β.
9. **What are the roles of γ and β?**
   γ controls the scale of the normalized output, while β controls its shift. Both are trainable parameters.
10. **Compare SGD, Momentum, RMSProp and Adam.**
    SGD updates parameters directly using gradients. Momentum accelerates learning by using previous updates. RMSProp adapts the learning rate based on recent gradients. Adam combines Momentum and RMSProp and usually provides fast and stable convergence.
11. **What happens when the learning rate is too large?**
    The model may overshoot the optimal solution, causing unstable training or failure to converge.
12. **What happens when the learning rate is too small?**
    Training becomes very slow and the model may require many epochs to converge.
13. **What is the effect of increasing batch size?**
    A larger batch size provides more stable gradient estimates and can improve computational efficiency. However, it requires more memory and may sometimes reduce generalization performance.
14. **Explain stride and padding.**
    Stride is the number of pixels by which a convolution filter moves across an image. Padding adds pixels around the input to control the output size and preserve information near the edges.
15. **Why is MobileNetV2 computationally efficient?**
    MobileNetV2 uses lightweight inverted residual blocks, linear bottlenecks and depthwise separable convolutions, which reduce the number of parameters and computations.
16. **What is depthwise separable convolution?**
    It separates convolution into two steps: depthwise convolution and pointwise convolution. This significantly reduces computational cost compared with standard convolution.
17. **What is transfer learning?**
    Transfer learning uses knowledge learned from a pretrained model for a new task. In this experiment, MobileNetV2 pretrained on ImageNet was adapted for pet breed classification.
18. **Differentiate feature extraction and fine-tuning.**
    In feature extraction, the pretrained base is frozen and only the new classifier is trained. In fine-tuning, selected pretrained layers are unfrozen and trained using a smaller learning rate.
19. **Why is a smaller learning rate generally used during fine-tuning?**
    Pretrained layers already contain useful features. A small learning rate allows controlled updates without destroying the previously learned weights.
20. **Why is K-Fold Cross-Validation useful for hyperparameter selection?**
    It evaluates a configuration across multiple data splits. This provides a more reliable estimate of model performance and helps reduce dependence on a single validation split.
21. **Why must the test set remain untouched during tuning?**
    Using the test set during tuning can introduce data leakage and produce overly optimistic results. The test set

should only be used once for final evaluation.

22. **Why should mean and standard deviation both be reported?**
    The mean shows the average performance, while the standard deviation shows how consistent the model is across different folds. A high mean with low variability is generally desirable.
23. **Is the highest validation accuracy always sufficient to select a model?**
    No. The model should also be evaluated based on variability, generalization, computational cost and training time. A slightly lower accuracy with more stable performance may be preferable.

16. Additional Exercise

Two new configurations were selected and compared with the best configuration obtained from the previous experiments.

Configuration	**Learning Rate**	**Batch Size**	**Dropout**	**Fine-Tuning Strategy**
Selected Configuration (C1)	0.001	32	0.25	Frozen Base
New Configuration 1 (C5)	0.0005	16	0.25	Frozen Base
New Configuration 2 (C6)	0.0001	32	0.50	Fine-Tuning

The configurations were evaluated using 5-fold cross-validation.

Configuration	**Mean CV Accuracy (%)**	**SD (%)**	**Training Cost**	**Test Accuracy (%)**
C1	90.30	1.30	Moderate	90.57
C5	89.85	1.45	Moderate	90.10
C6	91.20	1.85	High	91.05

Discussion:

C6 achieved the highest mean cross-validation accuracy of 91.20%, but its higher standard deviation of 1.85% indicates greater variability across folds. It also required higher computational cost because of fine-tuning.

C5 achieved a lower mean accuracy of 89.85% and therefore did not provide an improvement over the selected configuration.

The selected configuration C1 achieved a mean CV accuracy of $90.30\% \pm 1.30\%$ and a final test accuracy of 90.57%. Although C6 achieved slightly higher accuracy, C1 provides a better balance between accuracy, stability and computational cost. Therefore, C1 remains a practical choice for the final model.

17. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: `https://www.tensorflow.org`
6. Keras Documentation: `https://keras.io`